SRM ALPHA 300
CATEGORY B

OCTOBER 2026 DSA EXAMINATION BANK
10 QUESTION PAPER SETS

Question Papers · Sets 01 – 10

Python 3 · 5 questions per set · 3 Easy + 2 Medium · 100 marks · 120 minutes

Syllabus source: SRM Alpha 300 · Category B · October 2026 DSA Plan (Arrays + Sorting + Binary Search)

All solutions use only permitted primitives — no built-in sorting, bisect, heapq, collections or itertools shortcuts.

TABLE OF CONTENTS

SET 01 — QUESTION PAPER 4

Question 1 (Easy) — Gift Card Pairing at Checkout 4
Question 2 (Easy) — First Duplicate Ticket Scan 5
Question 3 (Easy) — Second Highest Sensor Reading 5
Question 4 (Medium) — Warehouse Balance Point 6
Question 5 (Medium) — Missing Roll Numbers 6

SET 02 — QUESTION PAPER 8

Question 1 (Easy) — Campus Election Winner 8
Question 2 (Easy) — First Unique Visitor 9
Question 3 (Easy) — Longest Attendance Streak 9
Question 4 (Medium) — Daily Sales Range Reports 10
Question 5 (Medium) — Delivery Route Rotation 10

SET 03 — QUESTION PAPER 12

Question 1 (Easy) — Mirror Coupon Code 12
Question 2 (Easy) — Voice Assistant Vowel Swap 13
Question 3 (Easy) — Out-of-Stock Items to the End 13
Question 4 (Medium) — Merge Two Warehouse Stock Lists 14
Question 5 (Medium) — Seat Booking Cleanup 14

SET 04 — QUESTION PAPER 16

Question 1 (Easy) — Peak Traffic Window 16
Question 2 (Easy) — Repeated Login Alert 17
Question 3 (Easy) — Strong Average Days 17
Question 4 (Medium) — Plagiarism Pattern Scanner 18
Question 5 (Medium) — Longest Unique Scan Run 18

SET 05 — QUESTION PAPER 20

Question 1 (Easy) — Mock Test Ranking Passes 20
Question 2 (Easy) — Fastest Deliveries Podium 21
Question 3 (Easy) — Nearly Sorted Event Log 21
Question 4 (Medium) — Leaderboard Disorder Count 22
Question 5 (Medium) — K-th Highest Salary 22

SET 06 — QUESTION PAPER 24

Question 1 (Easy) — Exam Marks Ranking by Counting 24
Question 2 (Easy) — Warehouse Bin Sorting 25
Question 3 (Easy) — Combined Attendee List 25
Question 4 (Medium) — Product Popularity Ordering 26
Question 5 (Medium) — Sensor Readings with Heavy Duplicates 26

SET 07 — QUESTION PAPER 28

Question 1 (Easy) — Library Catalogue Lookup 28
Question 2 (Easy) — Student ID Insertion Point 29
Question 3 (Easy) — First Failing Nightly Build 29
Question 4 (Medium) — Transaction Time Window 30
Question 5 (Medium) — Trail Elevation Search 30

SET 08 — QUESTION PAPER 32

Question 1 (Easy) — Square Tile Floor Planner 32
Question 2 (Easy) — Seat Chart Lookup 33
Question 3 (Easy) — Earliest Slot After Shift Change 33
Question 4 (Medium) — Rotated Log Lookup 34
Question 5 (Medium) — Cargo Shipping Capacity 34

SET 09 — QUESTION PAPER.....36
Question 1 (Easy) — Budget Pairing in a Sorted Price List.....36
Question 2 (Easy) — Judges' Trimmed Mean.....37
Question 3 (Easy) — Common Club Members.....37
Question 4 (Medium) — Booking Overlap Consolidation.....38
Question 5 (Medium) — Insert a Booking into the Calendar.....39
SET 10 — QUESTION PAPER.....40
Question 1 (Easy) — Sensor Deviation Magnitudes.....40
Question 2 (Easy) — Price Tier Floor and Ceiling.....41
Question 3 (Easy) — One-Trade Stock Profit.....41
Question 4 (Medium) — Zero-Balance Triplets.....42
Question 5 (Medium) — Textbook Allocation for Reading Groups.....42

SET 01 — QUESTION PAPER

SRM ALPHA 300 — CATEGORY B

OCTOBER 2026 DSA EXAMINATION

SET 01

Set number	01 of 10
Programming language	Python 3
Total questions	5
Difficulty distribution	3 Easy + 2 Medium
Maximum marks	100 (Questions 1–3: 16 marks each; Questions 4–5: 26 marks each)
Suggested duration	120 minutes

Examination instructions

- Attempt all five questions.
- Write complete Python 3 solutions.
- Explain the approach used.
- State time and auxiliary-space complexity.
- Follow the source PDF's restrictions on shortcut libraries.
- Use the specified input and output formats.
- Do not hard-code the sample output.
- Handle edge cases within the stated constraints.

QUESTION 1 — EASY (16 marks)

Title: Gift Card Pairing at Checkout

Difficulty: Easy

Scenario: An online store lets customers pay with a gift card whose balance must be used up exactly. At checkout, the system must find two different items in the customer's cart whose prices add up to exactly the gift-card balance.

Problem statement: Given the prices of the n items in a cart (in the order they were added) and the gift-card balance T, find two different positions i and j (0-based, i < j) such that price[i] + price[j] = T. The input guarantees that exactly one such pair of positions exists. The same position cannot be used twice.

Input format:

Line 1: integer n, the number of items.

Line 2: n space-separated integers, the item prices.

Line 3: integer T, the gift-card balance.

Output format:

Print two space-separated integers i and j (0-based, i < j) on one line.

Constraints:

- 2 ≤ n ≤ 10^5
- 1 ≤ price[i] ≤ 10^9
- 1 ≤ T ≤ 2 × 10^9
- Exactly one pair of positions (i, j) satisfies the condition

Sample input:

```
6
120 45 80 200 35 150
235
```

Sample output:

```
3 4
```

QUESTION 2 — EASY (16 marks)

Title: First Duplicate Ticket Scan

Difficulty: Easy

Scenario: At the gate of a campus fest, a scanner records each ticket ID as people walk in. A ticket must only be used once. Security wants to know the ID of the first scan that turns out to be a repeat of an earlier scan, so they can stop that person.

Problem statement: Given the sequence of n scanned ticket IDs in scan order, print the ID of the earliest scan whose value has already appeared at an earlier position. If no ID repeats, print -1. (Note: the answer is determined by the position of the second occurrence, not the first occurrence.)

Input format:

Line 1: integer n, the number of scans.

Line 2: n space-separated integers, the ticket IDs in scan order.

Output format:

Print a single integer: the ticket ID of the first repeated scan, or -1.

Constraints:

- 1 ≤ n ≤ 10^5
- 1 ≤ ID ≤ 10^9

Sample input:

```
8
4021 3377 5120 3377 4021 9001 5120 7
```

Sample output:

```
3377
```

QUESTION 3 — EASY (16 marks)

Title: Second Highest Sensor Reading

Difficulty: Easy

Scenario: A weather station records n temperature readings in one day. The station manager wants the second-highest distinct reading, to compare against the day's peak and judge how sharp the peak was.

Problem statement: Given n integer readings (possibly negative and possibly repeated), print the second-largest distinct value. Distinct means equal values count once: for the readings 9 9 5 the second-largest distinct value is 5. If fewer than two distinct values exist, print the word None.

Input format:

Line 1: integer n.

Line 2: n space-separated integers, the readings.

Output format:

Print one integer: the second-largest distinct reading, or the word None if it does not exist.

Constraints:

- 1 ≤ n ≤ 10^5
- -10^9 ≤ reading ≤ 10^9
- Do not use built-in max, min, sum or sorting.

Sample input:

```
9
31 28 35 35 29 -4 35 28 30
```

Sample output:

```
31
```

QUESTION 4 — MEDIUM (26 marks)

Title: Warehouse Balance Point

Difficulty: Medium

Scenario: A warehouse has n shelves in one aisle. Each shelf records a net stock change for the day (positive for received goods, negative for returns and write-offs). The supervisor wants a 'balance shelf': a shelf where the total change on all shelves strictly to its left equals the total change on all shelves strictly to its right, so the aisle can be audited as two balanced halves.

Problem statement: Given the n net stock changes, find the leftmost index p (0-based) such that the sum of the elements with index smaller than p equals the sum of the elements with index larger than p. An empty side has sum 0. Print p, or -1 if no such index exists. The element at p itself belongs to neither side.

Input format:

Line 1: integer n.

Line 2: n space-separated integers, the net stock changes.

Output format:

Print one integer: the leftmost balance index, or -1.

Constraints:

- 1 ≤ n ≤ 10^5
- -1000 ≤ change ≤ 1000

Sample input:

```
6
3 -2 5 4 1 5
```

Sample output:

```
3
```

QUESTION 5 — MEDIUM (26 marks)

Title: Missing Roll Numbers

Difficulty: Medium

Scenario: A class has n students with roll numbers 1 to n. The attendance sheet has exactly n entries, but because of copy-paste mistakes some roll numbers were written more than once, so some students are not on the sheet at all. The teacher wants the list of roll numbers that are missing from the sheet.

Problem statement: Given n and an array of n integers, each between 1 and n inclusive, print all integers in the range 1..n that do not appear in the array, in increasing order, separated by single spaces. If every roll number appears, print the word None. The array may be modified; try to use no extra array.

Input format:

Line 1: integer n.

Line 2: n space-separated integers, the roll numbers on the sheet.

Output format:

One line containing the missing roll numbers in increasing order separated by spaces, or None.

Constraints:

- 1 ≤ n ≤ 10^5
- 1 ≤ entry ≤ n

Sample input:

```
8
4 3 2 7 8 2 3 1
```

Sample output:

```
5 6
```

SET 02 — QUESTION PAPER

SRM ALPHA 300 — CATEGORY B

OCTOBER 2026 DSA EXAMINATION

SET 02

Set number	02 of 10
Programming language	Python 3
Total questions	5
Difficulty distribution	3 Easy + 2 Medium
Maximum marks	100 (Questions 1–3: 16 marks each; Questions 4–5: 26 marks each)
Suggested duration	120 minutes

Examination instructions

- Attempt all five questions.
- Write complete Python 3 solutions.
- Explain the approach used.
- State time and auxiliary-space complexity.
- Follow the source PDF's restrictions on shortcut libraries.
- Use the specified input and output formats.
- Do not hard-code the sample output.
- Handle edge cases within the stated constraints.

QUESTION 1 — EASY (16 marks)

Title: Campus Election Winner

Difficulty: Easy

Scenario: In a student-council election, every ballot carries a single candidate ID. The election rules say a candidate wins outright only by receiving strictly more than half of all ballots, and the returning officer has already verified that such a candidate exists in this round.

Problem statement: Given the n candidate IDs written on the ballots, find and print the ID that appears strictly more than ⌊n/2⌋ times. The input guarantees such an ID exists and is unique.

Input format:

Line 1: integer n, the number of ballots.

Line 2: n space-separated integers, the candidate IDs.

Output format:

Print one integer: the winning candidate ID.

Constraints:

- 1 ≤ n ≤ 10^5
- 1 ≤ ID ≤ 10^9
- A candidate with more than ⌊n/2⌋ ballots always exists

Sample input:

```
9
3 2 3 3 1 3 2 3 3
```

Sample output:

```
3
```

QUESTION 2 — EASY (16 marks)

Title: First Unique Visitor

Difficulty: Easy

Scenario: A website logs visitor IDs in the order they open the landing page. The marketing team wants to follow up with the earliest visitor who visited exactly once in the whole log (a one-time visitor).

Problem statement: Given the log of n visitor IDs, print the ID of the first visitor (by position in the log) whose ID occurs exactly once in the entire log. If every ID occurs at least twice, print -1.

Input format:

Line 1: integer n.

Line 2: n space-separated positive integers, the visitor IDs in log order.

Output format:

Print one integer: the first non-repeating ID, or -1.

Constraints:

- 1 ≤ n ≤ 10^5
- 1 ≤ ID ≤ 10^9

Sample input:

```
10
17 23 17 8 23 42 8 5 42 17
```

Sample output:

```
5
```

QUESTION 3 — EASY (16 marks)

Title: Longest Attendance Streak

Difficulty: Easy

Scenario: A college tracks daily attendance counts for a class over n days. A 'good day' is a day with at least T students present. The department rewards classes with long unbroken runs of good days.

Problem statement: Given the n daily attendance counts and the threshold T, print the length of the longest run of consecutive days in which every day has attendance ≥ T. If there is no good day, print 0.

Input format:

Line 1: two integers n and T.

Line 2: n space-separated integers, the daily attendance counts.

Output format:

Print one integer: the longest streak length.

Constraints:

- 1 ≤ n ≤ 10^5
- 1 ≤ T ≤ 10^4
- 0 ≤ attendance ≤ 10^4

Sample input:

```
10 30
28 35 40 31 30 12 33 36 29 41
```

Sample output:

```
4
```

QUESTION 4 — MEDIUM (26 marks)

Title: Daily Sales Range Reports

Difficulty: Medium

Scenario: A shop keeps the sales total for each of n days. The owner repeatedly asks for the total sales between two days (inclusive), for example 'what did we sell from day 3 to day 17?'. The daily numbers never change, but many queries arrive, so each answer must be fast.

Problem statement: Given the n daily sales and q queries, each query being two 0-based indices l and r (l ≤ r), print the sum of sales[l..r] for every query, one answer per line, in query order.

Input format:

Line 1: integers n and q.

Line 2: n space-separated integers, the daily sales.

Next q lines: two integers l and r per line.

Output format:

Print q lines; line k holds the answer to the k-th query.

Constraints:

- 1 ≤ n, q ≤ 10^5
- 0 ≤ sales[i] ≤ 10^4
- 0 ≤ l ≤ r ≤ n − 1
- Built-in sum() and itertools.accumulate are not allowed

Sample input:

```
8 4
5 3 8 2 7 1 9 4
0 2
3 5
1 7
4 4
```

Sample output:

```
16
10
34
7
```

QUESTION 5 — MEDIUM (26 marks)

Title: Delivery Route Rotation

Difficulty: Medium

Scenario: A courier company schedules n delivery stops for a driver. To balance workload, each new day's route is the previous day's route rotated to the right by k stops: the last k stops move to the front, and all other stops shift right keeping their order. Memory on the driver's handheld device is tiny, so the rotation must be done inside the same array without allocating another full-size list.

Problem statement: Given the route (stop IDs in order) and the integer k, print the route after rotating it to the right by k positions. The value k may be 0, equal to n or larger than n; rotating by n returns the original order. Slicing tricks such as a[k:] + a[:k] and list reversal helpers are not allowed; reverse with explicit loops and swaps.

Input format:

Line 1: two integers n and k.

Line 2: n space-separated integers, the stop IDs.

Output format:

Print the rotated route as n space-separated integers on one line.

Constraints:

- 1 ≤ n ≤ 10^5
- 0 ≤ k ≤ 10^9
- 1 ≤ stop ID ≤ 10^9

Sample input:

```
7 3
1 2 3 4 5 6 7
```

Sample output:

```
5 6 7 1 2 3 4
```

SET 03 — QUESTION PAPER

SRM ALPHA 300 — CATEGORY B

OCTOBER 2026 DSA EXAMINATION

SET 03

Set number	03 of 10
Programming language	Python 3
Total questions	5
Difficulty distribution	3 Easy + 2 Medium
Maximum marks	100 (Questions 1–3: 16 marks each; Questions 4–5: 26 marks each)
Suggested duration	120 minutes

Examination instructions

- Attempt all five questions.
- Write complete Python 3 solutions.
- Explain the approach used.
- State time and auxiliary-space complexity.
- Follow the source PDF's restrictions on shortcut libraries.
- Use the specified input and output formats.
- Do not hard-code the sample output.
- Handle edge cases within the stated constraints.

QUESTION 1 — EASY (16 marks)

Title: Mirror Coupon Code

Difficulty: Easy

Scenario: A shop issues 'mirror coupons': a promotion phrase is valid for the lucky-mirror discount only if it reads the same forwards and backwards. Customers type the phrase with spaces, punctuation and mixed capitals, and all of these must be ignored when checking.

Problem statement: Given a line of text, decide whether it is a palindrome after ignoring every character that is not a letter or digit and treating uppercase and lowercase letters as equal. Print YES if it is a palindrome and NO otherwise. A phrase that contains no letters or digits at all counts as a palindrome. Slicing and reversal shortcuts (s[::-1], reversed) are not allowed.

Input format:

A single line containing the phrase (letters, digits, spaces and punctuation).

Output format:

Print YES or NO.

Constraints:

- 1 ≤ length of the line ≤ 2 × 10^5
- The line contains printable ASCII characters

Sample input:

```
Was it a car or a cat I saw?
```

Sample output:

```
YES
```

QUESTION 2 — EASY (16 marks)

Title: Voice Assistant Vowel Swap

Difficulty: Easy

Scenario: A speech-to-text test tool deliberately garbles a phrase to test a spelling-correction feature: it keeps every consonant and space where it is, but reverses the order in which the vowels appear along the phrase.

Problem statement: Given a phrase made of English letters and single spaces, reverse the order of its vowels (a, e, i, o, u in either case) while all other characters stay in their original positions. Print the resulting phrase. Vowel letters keep their own case; only their positions change.

Input format:

A single line containing the phrase (letters and spaces; no leading or trailing spaces).

Output format:

Print the transformed phrase on one line.

Constraints:

- 1 ≤ length ≤ 2 × 10^5
- Characters are English letters or spaces

Sample input:

```
Campus Library
```

Sample output:

```
Campis Lubrary
```

QUESTION 3 — EASY (16 marks)

Title: Out-of-Stock Items to the End

Difficulty: Easy

Scenario: A warehouse dashboard shows the stock count of n products in catalogue order. Products with count 0 are out of stock and should be pushed to the end of the list so that available products appear first, but the relative order of the available products must stay exactly as in the catalogue. The dashboard runs on a small device, so the rearrangement should happen inside the same array.

Problem statement: Given n stock counts, move every 0 to the end while keeping the relative order of the non-zero counts. Print the rearranged counts. Try to do it in a single pass without creating a second array.

Input format:

Line 1: integer n.

Line 2: n space-separated integers, the stock counts.

Output format:

Print the rearranged counts on one line separated by spaces.

Constraints:

- 1 ≤ n ≤ 10^5
- 0 ≤ count ≤ 10^9

Sample input:

```
9
0 14 0 3 7 0 0 9 5
```

Sample output:

```
14 3 7 9 5 0 0 0 0
```

QUESTION 4 — MEDIUM (26 marks)

Title: Merge Two Warehouse Stock Lists

Difficulty: Medium

Scenario: Warehouse A holds its item codes in a sorted list that already has spare empty slots at the end (marked 0), exactly enough to hold all item codes from warehouse B, whose own list is also sorted. The system must merge B's codes into A's list so that A's list becomes one sorted list, without allocating a third list.

Problem statement: You are given m, n, an array A of length m+n whose first m entries are sorted (non-decreasing) and whose last n entries are placeholder zeros, and a sorted array B of n entries. Merge B into A so that A holds all m+n values in non-decreasing order, and print A. The placeholders are not real data; they must not appear in the result unless a real value equals 0. Do not use built-in sorting.

Input format:

Line 1: integers m and n.

Line 2: m+n integers (the first m are sorted real values, the last n are 0 placeholders).

Line 3: n integers, the sorted array B (an empty line if n = 0).

Output format:

Print the m+n merged values on one line, separated by spaces.

Constraints:

- 0 ≤ m, n ≤ 10^5, m + n ≥ 1
- -10^9 ≤ value ≤ 10^9
- Both input lists are sorted in non-decreasing order

Sample input:

```
4 3
1 4 7 10 0 0 0
2 5 8
```

Sample output:

```
1 2 4 5 7 8 10
```

QUESTION 5 — MEDIUM (26 marks)

Title: Seat Booking Cleanup

Difficulty: Medium

Scenario: A theatre's booking table lists seat IDs in sorted order, one entry per booking. A seat may legally be booked at most twice (for the matinee and the evening show), but a software bug created extra duplicate entries. The cleanup tool must keep at most two entries per seat, preserve the order, and compact the table in place.

Problem statement: Given a sorted (non-decreasing) array of n seat IDs, remove extra duplicates in place so that every distinct value appears at most twice, keeping the first two occurrences. Print k, the length of the cleaned list, and then the first k values of the modified array.

Input format:

Line 1: integer n.

Line 2: n space-separated integers in non-decreasing order.

Output format:

Line 1: the integer k.

Line 2: the k remaining values separated by spaces.

Constraints:

- 1 ≤ n ≤ 10^5
- 1 ≤ seat ID ≤ 10^9
- The array is sorted; no extra array may be created for the final answer

Sample input:

```
10
1 1 1 2 2 3 3 3 3 4
```

Sample output:

```
7
1 1 2 2 3 3 4
```

SET 04 — QUESTION PAPER

SRM ALPHA 300 — CATEGORY B

OCTOBER 2026 DSA EXAMINATION

SET 04

Set number	04 of 10
Programming language	Python 3
Total questions	5
Difficulty distribution	3 Easy + 2 Medium
Maximum marks	100 (Questions 1–3: 16 marks each; Questions 4–5: 26 marks each)
Suggested duration	120 minutes

Examination instructions

- Attempt all five questions.
- Write complete Python 3 solutions.
- Explain the approach used.
- State time and auxiliary-space complexity.
- Follow the source PDF's restrictions on shortcut libraries.
- Use the specified input and output formats.
- Do not hard-code the sample output.
- Handle edge cases within the stated constraints.

QUESTION 1 — EASY (16 marks)

Title: Peak Traffic Window

Difficulty: Easy

Scenario: A news website records the number of visitors for each of n consecutive hours. The operations team wants to schedule server upgrades away from the busiest period, defined as the k consecutive hours with the highest total visitors.

Problem statement: Given the hourly visitor counts and the window length k, print the largest possible total visitors over any k consecutive hours. Do not use built-in sum(), max() or slicing inside the loop; maintain the window total incrementally.

Input format:

Line 1: integers n and k.

Line 2: n space-separated integers, the hourly visitor counts.

Output format:

Print one integer: the maximum window total.

Constraints:

- 1 ≤ k ≤ n ≤ 10^5
- -10^4 ≤ count ≤ 10^4 (negative values represent corrections)

Sample input:

```
9 3
4 2 10 1 3 9 6 5 7
```

Sample output:

```
20
```

QUESTION 2 — EASY (16 marks)

Title: Repeated Login Alert

Difficulty: Easy

Scenario: A bank's login-monitoring system receives a stream of user IDs, one per login event. If the same user ID logs in twice within k events of each other (positions differing by at most k), the system raises a suspicious-activity alert.

Problem statement: Given the n user IDs in event order and the integer k, print YES if there exist two different positions i < j with the same ID and j − i ≤ k; otherwise print NO.

Input format:

Line 1: integers n and k.

Line 2: n space-separated integers, the user IDs.

Output format:

Print YES or NO.

Constraints:

- 1 ≤ n ≤ 10^5
- 1 ≤ k ≤ 10^5
- 1 ≤ ID ≤ 10^9

Sample input:

```
8 2
7 12 5 7 12 9 12 4
```

Sample output:

```
YES
```

QUESTION 3 — EASY (16 marks)

Title: Strong Average Days

Difficulty: Easy

Scenario: A school records the number of students present each day for n days. A 'strong period' is any k consecutive days whose average attendance is at least T. The principal wants to know how many strong periods (windows) exist.

Problem statement: Given n daily attendance counts, an integer k and a threshold T, print the number of windows of exactly k consecutive days whose average is at least T. To avoid fractions, a window qualifies when its sum is at least k·T. Overlapping windows are all counted.

Input format:

Line 1: integers n, k and T.

Line 2: n space-separated integers, the daily counts.

Output format:

Print one integer: the number of qualifying windows.

Constraints:

- 1 ≤ k ≤ n ≤ 10^5
- 0 ≤ count ≤ 10^4
- 1 ≤ T ≤ 10^4

Sample input:

```
8 3 4
2 2 2 2 5 5 5 8
```

Sample output:

```
3
```

QUESTION 4 — MEDIUM (26 marks)

Title: Plagiarism Pattern Scanner

Difficulty: Medium

Scenario: A plagiarism checker looks for 'scrambled copies' of a short keyword inside a long document: a stretch of text that uses exactly the same letters as the keyword, in any order. It needs the starting position of every such stretch.

Problem statement: Given a text s and a keyword p (both lowercase English letters), print every index i (0-based, increasing) such that the substring s[i .. i+|p|−1] is an anagram of p, i.e. has the same multiset of letters. Print the word None if there is no such index. Overlapping occurrences are all reported.

Input format:

Line 1: the string s.

Line 2: the string p.

Output format:

One line with the indices separated by single spaces, or None.

Constraints:

- 1 ≤ |s|, |p| ≤ 10^5
- Only lowercase letters a–z
- Counter, sorted() and slicing in the loop are not allowed

Sample input:

```
cbaebabacd
abc
```

Sample output:

```
0 6
```

QUESTION 5 — MEDIUM (26 marks)

Title: Longest Unique Scan Run

Difficulty: Medium

Scenario: A conveyor belt scanner reads an item code for every parcel passing by. Quality control wants the longest stretch of consecutive parcels in which no code appears twice, because such stretches can be audited as 'clean' batches.

Problem statement: Given the sequence of n scanned item codes, print the length of the longest contiguous segment in which all codes are different.

Input format:

Line 1: integer n.

Line 2: n space-separated integers, the item codes.

Output format:

Print one integer: the length of the longest segment with no repeated code.

Constraints:

- 1 ≤ n ≤ 10^5
- 1 ≤ code ≤ 10^9

Sample input:

```
10
12 7 9 4 5 3 12 6 8 2
```

Sample output:

```
9
```

SET 05 — QUESTION PAPER

SRM ALPHA 300 — CATEGORY B

OCTOBER 2026 DSA EXAMINATION

SET 05

Set number	05 of 10
Programming language	Python 3
Total questions	5
Difficulty distribution	3 Easy + 2 Medium
Maximum marks	100 (Questions 1–3: 16 marks each; Questions 4–5: 26 marks each)
Suggested duration	120 minutes

Examination instructions

- Attempt all five questions.
- Write complete Python 3 solutions.
- Explain the approach used.
- State time and auxiliary-space complexity.
- Follow the source PDF's restrictions on shortcut libraries.
- Use the specified input and output formats.
- Do not hard-code the sample output.
- Handle edge cases within the stated constraints.

QUESTION 1 — EASY (16 marks)

Title: Mock Test Ranking Passes

Difficulty: Easy

Scenario: After a mock test, a teacher sorts the n marks of a small group in ascending order by hand using the 'bubble' method: she repeatedly walks along the list, swapping any two neighbouring marks that are in the wrong order. She wants to know how many walks (passes) were actually needed, meaning passes in which at least one swap happened, to judge how disordered the list was.

Problem statement: Implement bubble sort on the n marks (ascending). In each pass compare neighbouring pairs from the left end; swap a pair only if the left mark is strictly greater than the right one. Print the sorted marks on the first line and, on the second line, the number of productive passes, i.e. passes in which at least one swap was performed. Built-in sorting is not allowed.

Input format:

Line 1: integer n.

Line 2: n space-separated integers, the marks.

Output format:

Line 1: the sorted marks separated by spaces.

Line 2: the number of passes that performed at least one swap.

Constraints:

- 1 ≤ n ≤ 2000
- 0 ≤ mark ≤ 100

Sample input:

```
5
5 1 4 2 8
```

Sample output:

```
1 2 4 5 8
2
```

QUESTION 2 — EASY (16 marks)

Title: Fastest Deliveries Podium

Difficulty: Easy

Scenario: A courier company logs the delivery time in minutes of n completed orders. A weekly report needs only the k fastest deliveries, listed from fastest to slowest. Sorting the whole log would waste effort when k is small.

Problem statement: Given n delivery times and an integer k, print the k smallest times in non-decreasing order (duplicates count separately). Use selection-sort rounds: stop after k rounds instead of sorting everything. Built-in sorting, min() and heaps are not allowed.

Input format:

Line 1: integers n and k.

Line 2: n space-separated integers, the delivery times.

Output format:

Print k integers on one line separated by spaces, in non-decreasing order.

Constraints:

- 1 ≤ k ≤ n ≤ 3000
- 1 ≤ time ≤ 10^4

Sample input:

```
7 3
29 14 36 7 21 14 50
```

Sample output:

```
7 14 14
```

QUESTION 3 — EASY (16 marks)

Title: Nearly Sorted Event Log

Difficulty: Easy

Scenario: A server writes event timestamps almost in order, but network delays make a few events arrive late. Before storing the log, the system repairs the order with insertion sort and records how much repair work (element shifts) was needed as a measure of the delay problem.

Problem statement: Sort the n timestamps in non-decreasing order using insertion sort (take each element and move it left past every strictly greater element). Print the sorted list and the total number of single-position shifts performed, where moving one element one place to the right counts as one shift. Built-in sorting is not allowed.

Input format:

Line 1: integer n.

Line 2: n space-separated integers, the timestamps.

Output format:

Line 1: the sorted timestamps.

Line 2: the total number of shifts.

Constraints:

- 1 ≤ n ≤ 5000
- 0 ≤ timestamp ≤ 10^6

Sample input:

```
8
10 20 15 30 40 35 50 45
```

Sample output:

```
10 15 20 30 35 40 45 50
3
```

QUESTION 4 — MEDIUM (26 marks)

Title: Leaderboard Disorder Count

Difficulty: Medium

Scenario: A game server lists players' scores in the order the players finished their matches. A perfectly 'fair' queue would have scores in non-decreasing order. To measure unfairness, the analyst counts the number of pairs of players where the earlier finisher has a strictly higher score than the later finisher.

Problem statement: Given n scores in finishing order, count the pairs of positions (i, j) with i < j and score[i] > score[j]. Print the count. Equal scores do not form an inversion. Built-in sorting is not allowed. The answer can be large, so an $O(n^2)$ pair check will not be fast enough for the largest inputs.

Input format:

Line 1: integer n.

Line 2: n space-separated integers, the scores.

Output format:

Print one integer: the number of inversions.

Constraints:

- 1 ≤ n ≤ 10^5
- 0 ≤ score ≤ 10^9

Sample input:

```
6
5 3 8 6 2 7
```

Sample output:

```
7
```

QUESTION 5 — MEDIUM (26 marks)

Title: K-th Highest Salary

Difficulty: Medium

Scenario: An HR analyst has the monthly salaries of n employees (many people share salaries) and needs the k-th highest salary, counting duplicates as separate people, for a benefits-eligibility cut-off. Sorting everyone just to read one value is wasteful.

Problem statement: Given n salaries and an integer k, print the k-th largest element of the multiset: the value that would be at position k if the salaries were arranged in non-increasing order (k = 1 is the maximum). Duplicates count separately. Built-in sorting is not allowed; use partitioning.

Input format:

Line 1: integers n and k.

Line 2: n space-separated integers, the salaries.

Output format:

Print one integer: the k-th largest salary.

Constraints:

- 1 ≤ k ≤ n ≤ 10^5
- 1 ≤ salary ≤ 10^9

Sample input:

```
9 3
40 25 60 25 90 75 60 30 50
```

Sample output:

```
60
```

SET 06 — QUESTION PAPER

SRM ALPHA 300 — CATEGORY B

OCTOBER 2026 DSA EXAMINATION

SET 06

Set number	06 of 10
Programming language	Python 3
Total questions	5
Difficulty distribution	3 Easy + 2 Medium
Maximum marks	100 (Questions 1–3: 16 marks each; Questions 4–5: 26 marks each)
Suggested duration	120 minutes

Examination instructions

- Attempt all five questions.
- Write complete Python 3 solutions.
- Explain the approach used.
- State time and auxiliary-space complexity.
- Follow the source PDF's restrictions on shortcut libraries.
- Use the specified input and output formats.
- Do not hard-code the sample output.
- Handle edge cases within the stated constraints.

QUESTION 1 — EASY (16 marks)

Title: Exam Marks Ranking by Counting

Difficulty: Easy

Scenario: A university publishes the marks of n students for an exam out of 100. Every mark is a whole number from 0 to 100, but n can be very large (tens of thousands). The registrar wants all marks listed in ascending order as quickly as possible.

Problem statement: Sort the n marks in non-decreasing order using counting sort: count how many times each mark 0..100 occurs, then rebuild the list from the counts. Built-in sorting and Counter are not allowed.

Input format:

Line 1: integer n.

Line 2: n space-separated integers, the marks.

Output format:

Print the sorted marks on one line separated by spaces.

Constraints:

- 1 ≤ n ≤ 10^5
- 0 ≤ mark ≤ 100

Sample input:

```
10
78 92 65 78 100 0 65 92 78 55
```

Sample output:

```
0 55 65 65 78 78 78 92 92 100
```

QUESTION 2 — EASY (16 marks)

Title: Warehouse Bin Sorting

Difficulty: Easy

Scenario: A conveyor delivers parcels tagged 0 (fragile), 1 (standard) or 2 (heavy). Before loading, the parcels must be arranged so that all fragile ones come first, then standard, then heavy. The loading arm can only swap two parcels at a time and has no room for a second copy of the belt.

Problem statement: Given n tags, each 0, 1 or 2, rearrange them in place into non-decreasing order and print the result. Aim for a single left-to-right pass using three pointers, without a general-purpose sort.

Input format:

Line 1: integer n.

Line 2: n space-separated tags, each 0, 1 or 2.

Output format:

Print the rearranged tags on one line separated by spaces.

Constraints:

- 1 ≤ n ≤ 10^5
- Each tag is 0, 1 or 2

Sample input:

```
8
2 0 1 2 1 0 1 0
```

Sample output:

```
0 0 0 1 1 1 2 2
```

QUESTION 3 — EASY (16 marks)

Title: Combined Attendee List

Difficulty: Easy

Scenario: Two student clubs each keep a sorted list of the student IDs who registered for their own workshop (no ID repeated within one list). The organisers want one combined invitation list in ascending order where a student who registered in both clubs appears only once.

Problem statement: Given two strictly increasing lists A (m IDs) and B (n IDs), print the union of the two lists in strictly increasing order, with each ID printed once. If both lists are empty, print an empty line (this does not occur in the tests). Built-in sorting is not allowed; exploit the fact that both inputs are already sorted.

Input format:

Line 1: integers m and n.

Line 2: m space-separated integers (list A; empty line if m = 0).

Line 3: n space-separated integers (list B; empty line if n = 0).

Output format:

Print the merged distinct IDs on one line separated by spaces.

Constraints:

- 0 ≤ m, n ≤ 10^5, m + n ≥ 1
- 1 ≤ ID ≤ 10^9
- Each input list is strictly increasing

Sample input:

```
5 4
3 8 12 20 25
5 8 20 31
```

Sample output:

```
3 5 8 12 20 25 31
```

QUESTION 4 — MEDIUM (26 marks)

Title: Product Popularity Ordering

Difficulty: Medium

Scenario: An online store logs the product ID of every sale. A 'rarity report' lists the sales so that the least frequently sold products come first. Products sold equally often are listed with the larger product ID first. Each sale is listed individually, so a product sold three times appears three times in a row.

Problem statement: Given the n product IDs of the sales, output all n IDs arranged so that (1) IDs with smaller total frequency come first, and (2) among IDs with the same frequency, the larger ID comes first. All copies of one ID are consecutive. Built-in sorting, Counter and sorted() with a key are not allowed; write your own sorting routine with a custom comparison.

Input format:

Line 1: integer n.

Line 2: n space-separated integers, the product IDs.

Output format:

Print the n IDs in the required order on one line, separated by spaces.

Constraints:

- 1 ≤ n ≤ 10^5
- 1 ≤ ID ≤ 10^9

Sample input:

```
9
4 4 6 2 2 2 6 9 6
```

Sample output:

```
9 4 4 6 6 6 2 2 2
```

QUESTION 5 — MEDIUM (26 marks)

Title: Sensor Readings with Heavy Duplicates

Difficulty: Medium

Scenario: A factory sensor reports a quantised reading (a small integer level) every second, so a day's data contains many repeated values and only a few distinct ones. Analysts want the readings sorted in ascending order. A sort that treats equal readings badly would waste time on the duplicates, so the team wants a quicksort that groups all readings equal to the pivot together in one step.

Problem statement: Sort the n readings in non-decreasing order using quicksort with 3-way (Dutch-flag) partitioning: elements smaller than the pivot, equal to the pivot and larger than the pivot are separated, and only the smaller and larger parts are sorted further. Use an explicit stack of index ranges (no recursion) and choose the middle element of each range as pivot. Built-in sorting is not allowed.

Input format:

Line 1: integer n.

Line 2: n space-separated integers, the readings.

Output format:

Print the sorted readings on one line separated by spaces.

Constraints:

- 1 ≤ n ≤ 10^5
- 0 ≤ reading ≤ 10^9 (typically only a few distinct values)

Sample input:

```
10
3 1 3 2 3 1 2 3 1 3
```

Sample output:

```
1 1 1 2 2 3 3 3 3 3
```

SET 07 — QUESTION PAPER

SRM ALPHA 300 — CATEGORY B

OCTOBER 2026 DSA EXAMINATION

SET 07

Set number	07 of 10
Programming language	Python 3
Total questions	5
Difficulty distribution	3 Easy + 2 Medium
Maximum marks	100 (Questions 1–3: 16 marks each; Questions 4–5: 26 marks each)
Suggested duration	120 minutes

Examination instructions

- Attempt all five questions.
- Write complete Python 3 solutions.
- Explain the approach used.
- State time and auxiliary-space complexity.
- Follow the source PDF's restrictions on shortcut libraries.
- Use the specified input and output formats.
- Do not hard-code the sample output.
- Handle edge cases within the stated constraints.

QUESTION 1 — EASY (16 marks)

Title: Library Catalogue Lookup

Difficulty: Easy

Scenario: A university library keeps the book IDs on one shelf-row in strictly increasing order. The librarian's terminal must tell a student at which shelf position (0-based) a given book ID sits, or that the book is not on this row. Because there are thousands of books and many lookups per minute, scanning the row one book at a time is too slow.

Problem statement: Given a strictly increasing array of n book IDs and a query ID q, print the 0-based index of q in the array, or -1 if q does not occur. Use binary search with lo, hi and mid = lo + (hi − lo) // 2; do not use bisect or list.index.

Input format:

Line 1: integer n.
Line 2: n strictly increasing integers.
Line 3: integer q, the book ID to find.

Output format:

Print one integer: the index of q, or -1.

Constraints:

- 1 ≤ n ≤ 10^5
- -10^9 ≤ ID, q ≤ 10^9
- IDs are distinct and sorted in increasing order

Sample input:

```
9
102 118 130 145 160 177 190 204 230
177
```

Sample output:

```
5
```

QUESTION 2 — EASY (16 marks)

Title: Student ID Insertion Point

Difficulty: Easy

Scenario: A registrar keeps a sorted list of already-issued student IDs. When a new student arrives, the system must tell the clerk the position at which the new ID should be inserted to keep the list sorted. If the ID has already been issued, the system reports the position of that existing ID instead.

Problem statement: Given a strictly increasing array of n IDs and a target t, print the index of t if it is present; otherwise print the index where t would have to be inserted to keep the array sorted (the number of elements smaller than t). Use binary search; do not use bisect.

Input format:

Line 1: integer n.
Line 2: n strictly increasing integers.
Line 3: integer t.

Output format:

Print one integer between 0 and n inclusive.

Constraints:

- 1 ≤ n ≤ 10^5
- -10^9 ≤ ID, t ≤ 10^9
- IDs are distinct and sorted

Sample input:

```
6
10 20 30 45 60 80
50
```

Sample output:

```
4
```

QUESTION 3 — EASY (16 marks)

Title: First Failing Nightly Build

Difficulty: Easy

Scenario: A software team runs an automated test suite on each of n consecutive nightly builds. Once a bug is introduced, every later build also fails. The results are stored as a list: 0 for a passing build and 1 for a failing build. Re-running the full suite for every build is expensive, so the team wants to locate the first failing build with as few look-ups as possible.

Problem statement: The array has the form 0…0 1…1: some number (possibly zero) of 0s followed by 1s. Print the 1-based number of the first build with value 1, or -1 if every build passed. Use binary search.

Input format:

Line 1: integer n, the number of builds.

Line 2: n integers (0 or 1), non-decreasing, the test results in build order.

Output format:

Print one integer: the 1-based number of the first failing build, or -1.

Constraints:

- 1 ≤ n ≤ 10^5
- Each value is 0 or 1 and the sequence is non-decreasing

Sample input:

```
10
0 0 0 0 0 0 1 1 1 1
```

Sample output:

```
7
```

QUESTION 4 — MEDIUM (26 marks)

Title: Transaction Time Window

Difficulty: Medium

Scenario: A payment system stores transaction amounts of one merchant in sorted order (many transactions share the same amount, for example fixed-price tickets). An auditor wants to know where the block of transactions with a given amount starts and ends in the sorted list, to inspect that block.

Problem statement: Given a non-decreasing array of n amounts and a target x, print the 0-based index of the first occurrence of x and the index of the last occurrence, separated by a space. If x does not occur, print -1 -1. Use two binary searches; do not scan linearly and do not use bisect.

Input format:

Line 1: integer n.

Line 2: n integers in non-decreasing order.

Line 3: integer x.

Output format:

Print two integers: first index and last index, or -1 -1.

Constraints:

- 1 ≤ n ≤ 10^5
- -10^9 ≤ amount, x ≤ 10^9
- The array may contain duplicates

Sample input:

```
10
5 7 7 8 8 8 8 10 12 12
8
```

Sample output:

```
3 6
```

QUESTION 5 — MEDIUM (26 marks)

Title: Trail Elevation Search

Difficulty: Medium

Scenario: A hiking app records the altitude (in metres) at consecutive checkpoints along a trail that climbs strictly to a single summit and then descends strictly. Hikers ask 'at which checkpoint do I first reach altitude h?'. Because a checkpoint is read from a slow sensor log, the app wants to examine as few checkpoints as possible.

Problem statement: The array is a mountain: strictly increasing up to a unique peak, then strictly decreasing (n ≥ 3, the peak is neither the first nor the last element). Given a target h, print the smallest index whose value is h, or -1 if h does not occur. Do it in O(log n): find the peak by binary search, then binary-search the increasing part, and only if needed the decreasing part.

Input format:

Line 1: integer n.

Line 2: n integers forming a mountain.

Line 3: integer h.

Output format:

Print one integer: the smallest index with value h, or -1.

Constraints:

- 3 ≤ n ≤ 10^5
- -10^9 ≤ altitude, h ≤ 10^9
- Strictly increasing then strictly decreasing, with the peak strictly inside

Sample input:

```
9
2 5 9 14 20 17 11 6 1
11
```

Sample output:

```
6
```

SET 08 — QUESTION PAPER

SRM ALPHA 300 — CATEGORY B
OCTOBER 2026 DSA EXAMINATION
SET 08

Set number	08 of 10
Programming language	Python 3
Total questions	5
Difficulty distribution	3 Easy + 2 Medium
Maximum marks	100 (Questions 1–3: 16 marks each; Questions 4–5: 26 marks each)
Suggested duration	120 minutes

Examination instructions

- Attempt all five questions.
- Write complete Python 3 solutions.
- Explain the approach used.
- State time and auxiliary-space complexity.
- Follow the source PDF's restrictions on shortcut libraries.
- Use the specified input and output formats.
- Do not hard-code the sample output.
- Handle edge cases within the stated constraints.

QUESTION 1 — EASY (16 marks)

Title: Square Tile Floor Planner

Difficulty: Easy

Scenario: A contractor has x identical square floor tiles and wants to lay them as one large square block of s × s tiles. She wants the largest whole-number side s such that s × s tiles are available. The number of tiles can be astronomically large for industrial floors, so trying every side length one by one is too slow.

Problem statement: Given a non-negative integer x, print the largest integer s such that s × s ≤ x (the floor of the square root of x). Do not use the sqrt function, math.isqrt or the ** 0.5 trick; use binary search over the possible values of s.

Input format:

A single line containing the integer x.

Output format:

Print one integer s.

Constraints:

- 0 ≤ x ≤ 10^18

Sample input:

```
99
```

Sample output:

```
9
```

QUESTION 2 — EASY (16 marks)

Title: Seat Chart Lookup

Difficulty: Easy

Scenario: A cinema stores ticket numbers in an R × C seating chart. Each row is sorted from left to right, and the first ticket number of each row is larger than the last ticket number of the previous row. A usher scans a ticket and needs the seat (row and column) quickly.

Problem statement: Given the R × C matrix with the properties above and a ticket number t, print the 0-based row and column of t, separated by a space, or -1 -1 if t is not in the chart. Treat the matrix as a sorted 1-D array of R·C elements and binary search it using index arithmetic (row = idx // C, col = idx % C); do not flatten into a new list.

Input format:

Line 1: integers R and C.

Next R lines: C integers each (one row).

Last line: integer t.

Output format:

Print 'row col' or '-1 -1'.

Constraints:

- 1 ≤ R, C ≤ 300
- -10^9 ≤ value, t ≤ 10^9
- All values are distinct; each row is increasing and the first value of each row exceeds the last value of the previous row

Sample input:

```
3 4
1 3 5 7
10 11 16 20
23 30 34 60
16
```

Sample output:

```
1 2
```

QUESTION 3 — EASY (16 marks)

Title: Earliest Slot After Shift Change

Difficulty: Easy

Scenario: A hospital stores the start times (in minutes from midnight, all different) of appointment slots in increasing order, but after a shift change the rota was rotated: the list now starts in the middle of the original order and wraps around. The scheduler needs the earliest slot time, which is the minimum of the list.

Problem statement: The array is a sorted array of distinct integers that has been rotated by some unknown amount (possibly zero). Print the minimum element in O(log n) time using binary search; do not use min() or a full scan.

Input format:

Line 1: integer n.

Line 2: n distinct integers forming a rotated sorted array.

Output format:

Print one integer: the minimum value.

Constraints:

- 1 ≤ n ≤ 10^5
- -10^9 ≤ value ≤ 10^9
- All values are distinct

Sample input:

```
7
40 50 60 70 10 20 30
```

Sample output:

```
10
```

QUESTION 4 — MEDIUM (26 marks)

Title: Rotated Log Lookup

Difficulty: Medium

Scenario: A logging service stores sequence numbers in a circular buffer. Logically the numbers are sorted, but the buffer's start pointer has moved, so physically the array is a sorted array rotated by an unknown offset. Support engineers need to find the physical position of a given sequence number quickly.

Problem statement: Given a rotated sorted array of n distinct integers and a target t, print the index of t, or -1 if t is absent, in O(log n) time. Do not first locate the rotation point with a linear scan and do not use built-in search methods.

Input format:

Line 1: integer n.
Line 2: n distinct integers forming a rotated sorted array.
Line 3: integer t.

Output format:

Print one integer: the index of t, or -1.

Constraints:

- 1 ≤ n ≤ 10^5
- -10^9 ≤ value, t ≤ 10^9
- All values are distinct

Sample input:

```
8
15 18 22 31 3 6 9 12
9
```

Sample output:

```
6
```

QUESTION 5 — MEDIUM (26 marks)

Title: Cargo Shipping Capacity

Difficulty: Medium

Scenario: A freight company must ship n packages waiting in a fixed order at a depot. Each day the ship loads packages in that order (no reordering), up to its weight capacity, and sails. All packages must be delivered within D days. The company wants to charter the smallest possible ship capacity that makes this feasible.

Problem statement: Given the package weights in shipping order and D, print the minimum ship capacity C such that the packages can be shipped in order, each day carrying a consecutive group of packages whose total weight does not exceed C, using at most D days. The capacity must be at least the weight of the heaviest package.

Input format:

Line 1: integers n and D.

Line 2: n integers, the weights in shipping order.

Output format:

Print one integer: the minimum capacity.

Constraints:

- 1 ≤ D ≤ n ≤ 5 × 10^4
- 1 ≤ weight ≤ 500
- Do not use built-in max() or sum()

Sample input:

```
7 3
4 2 7 1 8 3 5
```

Sample output:

```
13
```

SET 09 — QUESTION PAPER

SRM ALPHA 300 — CATEGORY B
OCTOBER 2026 DSA EXAMINATION
SET 09

Set number	09 of 10
Programming language	Python 3
Total questions	5
Difficulty distribution	3 Easy + 2 Medium
Maximum marks	100 (Questions 1–3: 16 marks each; Questions 4–5: 26 marks each)
Suggested duration	120 minutes

Examination instructions

- Attempt all five questions.
- Write complete Python 3 solutions.
- Explain the approach used.
- State time and auxiliary-space complexity.
- Follow the source PDF's restrictions on shortcut libraries.
- Use the specified input and output formats.
- Do not hard-code the sample output.
- Handle edge cases within the stated constraints.

QUESTION 1 — EASY (16 marks)

Title: Budget Pairing in a Sorted Price List

Difficulty: Easy

Scenario: An online shop lists its products in non-decreasing order of price. A customer with a voucher of exact value T wants a two-item bundle whose prices add up to T exactly. The shop's server must answer instantly, even for catalogues with 10^5 items.

Problem statement: Given n prices in non-decreasing order and the target T, find the two different positions (1-based, i < j) whose prices sum to T and print them separated by a space. The input guarantees exactly one such pair of positions. The same position cannot be used twice.

Input format:

Line 1: integer n.
Line 2: n integers in non-decreasing order.
Line 3: integer T.

Output format:

Print two integers i and j (1-based, i < j).

Constraints:

- 2 ≤ n ≤ 10^5
- -10^9 ≤ price ≤ 10^9 (negative values represent refunds)
- -2×10^9 ≤ T ≤ 2×10^9
- Exactly one valid pair of positions exists
- Do not use extra memory proportional to n

Sample input:

```
7
3 8 12 19 25 31 40
44
```

Sample output:

```
4 5
```

QUESTION 2 — EASY (16 marks)

Title: Judges' Trimmed Mean

Difficulty: Easy

Scenario: In a talent contest n judges give integer scores. To reduce bias, the organisers discard the t lowest and the t highest scores and report the average of the remaining n − 2t scores, rounded to two decimals.

Problem statement: Given the n scores and an integer t with n − 2t ≥ 1, discard the t smallest and the t largest scores (equal scores are interchangeable) and print the arithmetic mean of the remaining scores formatted with exactly two digits after the decimal point. Implement your own sorting routine; built-in sort functions are not allowed.

Input format:

Line 1: integers n and t.

Line 2: n integers, the scores.

Output format:

Print the trimmed mean with exactly two decimals (use the format '%.2f').

Constraints:

- 1 ≤ n ≤ 10^5
- 0 ≤ t and n − 2t ≥ 1
- 0 ≤ score ≤ 100

Sample input:

```
10 1
8 9 7 10 6 9 5 8 9 10
```

Sample output:

```
8.25
```

QUESTION 3 — EASY (16 marks)

Title: Common Club Members

Difficulty: Easy

Scenario: Two campus clubs have separate sign-up sheets of student IDs, in no particular order, and some students signed up several times. The student-affairs office wants the list of students who appear on both sheets, with each student listed once, in increasing order of ID.

Problem statement: Given arrays A (m IDs) and B (n IDs), print the distinct IDs that occur in both arrays, in increasing order separated by spaces. If there is no common ID print None. Use a hash set for the membership test and your own sorting routine for the output; do not use set operators (&, |, -), sorted() or list.sort().

Input format:

Line 1: integers m and n.

Line 2: m integers (sheet A).

Line 3: n integers (sheet B).

Output format:

Common distinct IDs in increasing order on one line, or None.

Constraints:

- 1 ≤ m, n ≤ 10^5
- 1 ≤ ID ≤ 10^9

Sample input:

```
6 6
9 4 9 5 12 4
12 9 9 8 4 12
```

Sample output:

```
4 9 12
```

QUESTION 4 — MEDIUM (26 marks)

Title: Booking Overlap Consolidation

Difficulty: Medium

Scenario: A conference-room system receives n booking requests, each a closed time interval [start, end] in whole minutes, in arbitrary order. Overlapping bookings (and bookings that share an endpoint) are to be consolidated into single blocks so that facilities staff can see the busy periods of the room.

Problem statement: Merge all intervals that overlap or touch (two closed intervals merge when they share at least one point, e.g. [1,4] and [4,5] merge into [1,5], whereas [1,4] and [5,6] do not). Print the resulting disjoint intervals in increasing order of start, one interval per line as 'start end'. Use your own sort; built-in sort functions are not allowed.

Input format:

Line 1: integer n.

Next n lines: two integers start and end (start ≤ end).

Output format:

Print one merged interval per line as 'start end', ordered by start.

Constraints:

- 1 ≤ n ≤ 10^5
- 0 ≤ start ≤ end ≤ 10^9

Sample input:

```
6
8 10
1 3
2 6
15 18
17 20
6 7
```

Sample output:

```
1 7
8 10
15 20
```

QUESTION 5 — MEDIUM (26 marks)

Title: Insert a Booking into the Calendar

Difficulty: Medium

Scenario: A clinic's calendar for one doctor is a list of already consolidated bookings: closed intervals [start, end], sorted by start and pairwise disjoint (each start is strictly larger than the previous end). A new booking arrives and must be inserted; if it overlaps or touches existing bookings they must be merged into one block.

Problem statement: Given n sorted, pairwise disjoint intervals and one new interval, print the updated calendar: all intervals still sorted and disjoint, with the new interval merged into any intervals that share at least one point with it. Print one interval per line as 'start end'. The existing list is already sorted, so no sorting is needed.

Input format:

Line 1: integer n.

Next n lines: 'start end' for each existing interval.

Last line: 'start end' of the new interval.

Output format:

Print the resulting intervals, one per line as 'start end', in increasing order.

Constraints:

- 1 ≤ n ≤ 10^5
- 0 ≤ start ≤ end ≤ 10^9
- Existing intervals are sorted by start and pairwise disjoint (non-touching)

Sample input:

```
5
1 3
6 9
12 15
18 20
22 25
8 13
```

Sample output:

```
1 3
6 15
18 20
22 25
```

SET 10 — QUESTION PAPER

SRM ALPHA 300 — CATEGORY B
OCTOBER 2026 DSA EXAMINATION
SET 10

Set number	10 of 10
Programming language	Python 3
Total questions	5
Difficulty distribution	3 Easy + 2 Medium
Maximum marks	100 (Questions 1–3: 16 marks each; Questions 4–5: 26 marks each)
Suggested duration	120 minutes

Examination instructions

- Attempt all five questions.
- Write complete Python 3 solutions.
- Explain the approach used.
- State time and auxiliary-space complexity.
- Follow the source PDF's restrictions on shortcut libraries.
- Use the specified input and output formats.
- Do not hard-code the sample output.
- Handle edge cases within the stated constraints.

QUESTION 1 — EASY (16 marks)

Title: Sensor Deviation Magnitudes

Difficulty: Easy

Scenario: A calibration sensor reports deviations from a reference value, stored in increasing order, so negative deviations come first. Quality engineers need the squared deviations in increasing order as well, to see the smallest error magnitudes first.

Problem statement: Given a non-decreasing array of n integers (possibly negative), print the squares of its elements in non-decreasing order. Squaring and then calling a built-in sort is not allowed; exploit the fact that the input is already sorted.

Input format:

Line 1: integer n.

Line 2: n integers in non-decreasing order.

Output format:

Print n integers: the squares in non-decreasing order, separated by spaces.

Constraints:

- 1 ≤ n ≤ 10^5
- -10^4 ≤ value ≤ 10^4

Sample input:

```
7
-9 -4 -2 0 3 5 8
```

Sample output:

```
0 4 9 16 25 64 81
```

QUESTION 2 — EASY (16 marks)

Title: Price Tier Floor and Ceiling

Difficulty: Easy

Scenario: A shop's price list contains the available price tiers in increasing order (the same tier may be listed more than once). A customer states a budget q. The shop wants to quote the most expensive tier not above the budget (floor) and the cheapest tier not below the budget (ceiling), so that it can suggest the nearest options on either side.

Problem statement: Given a non-decreasing array of n prices and a query value q, print two values separated by a space: the floor (largest price ≤ q) and the ceiling (smallest price ≥ q). If a value does not exist print the word None in its place. Use binary search.

Input format:

Line 1: integer n.

Line 2: n integers in non-decreasing order.

Line 3: integer q.

Output format:

Print 'floor ceiling' where each is an integer or None.

Constraints:

- 1 ≤ n ≤ 10^5
- -10^9 ≤ price, q ≤ 10^9

Sample input:

```
6
10 20 30 40 50 60
35
```

Sample output:

```
30 40
```

QUESTION 3 — EASY (16 marks)

Title: One-Trade Stock Profit

Difficulty: Easy

Scenario: A trader has the closing price of a stock for each of n consecutive days. She may buy one share on some day and sell it on a strictly later day (at most one trade). She wants the maximum profit she could have made, or 0 if no profitable trade existed.

Problem statement: Given the daily prices, print the maximum value of price[j] − price[i] over all pairs i < j. If that maximum is negative (prices only fall) or n = 1, print 0 (no trade is made). Do not use built-in max or min.

Input format:

Line 1: integer n.

Line 2: n integers, the daily closing prices.

Output format:

Print one integer: the maximum profit (0 if none).

Constraints:

- 1 ≤ n ≤ 10^5
- 1 ≤ price ≤ 10^9

Sample input:

```
8
120 98 105 90 112 130 95 118
```

Sample output:

```
40
```

QUESTION 4 — MEDIUM (26 marks)

Title: Zero-Balance Triplets

Difficulty: Medium

Scenario: An auditor has a list of n ledger adjustments (positive for credits, negative for debits). She suspects fraud when three adjustments cancel exactly. She wants every distinct combination of three adjustments, taken from three different positions, that adds up to zero, without listing the same combination of values twice.

Problem statement: Given n integers, print all distinct triplets (x, y, z) with x ≤ y ≤ z and x + y + z = 0, where the three values come from three different positions. Triplets are distinct as value combinations (for example -1 0 1 is listed once even if -1 occurs twice). Print the triplets in lexicographic order, one per line as 'x y z'; if there are none print None. Use your own sort.

Input format:

Line 1: integer n.

Line 2: n integers.

Output format:

One triplet per line in lexicographic order, or None.

Constraints:

- 3 ≤ n ≤ 3000
- -10^5 ≤ value ≤ 10^5

Sample input:

```
8
-1 0 1 2 -1 -4 3 -2
```

Sample output:

```
-4 1 3
-2 -1 3
-2 0 2
-1 -1 2
-1 0 1
```

QUESTION 5 — MEDIUM (26 marks)

Title: Textbook Allocation for Reading Groups

Difficulty: Medium

Scenario: A library has n textbooks arranged on a shelf in a fixed order with known page counts. The professor splits the class into m reading groups. Each group is given a contiguous block of books from the shelf (no group is left without books, and no book is split). She wants the heaviest group workload, measured in total pages, to be as small as possible.

Problem statement: Given the page counts of the n books in shelf order and the number of groups m, assign contiguous blocks of books to the m groups so that every group receives at least one book and every

book is assigned. Print the minimum possible value of the largest total number of pages given to any group. If m > n it is impossible to give every group a book; print -1.

Input format:

Line 1: integers n and m.

Line 2: n integers, the page counts in shelf order.

Output format:

Print one integer: the minimised maximum pages, or -1 if m > n.

Constraints:

- 1 ≤ n ≤ 10^5
- 1 ≤ m ≤ 10^5
- 1 ≤ pages ≤ 10^4
- Do not use built-in max() or sum()

Sample input:

```
6 3
25 46 28 49 24 20
```

Sample output:

```
77
```